

Отчет по практической работе: Декодер формата DEFLATE (RFC 1951)

2026

1 Введение

Целью данной работы является реализация декодера потока данных, сжатых по алгоритму DEFLATE. Данный формат является промышленным стандартом и сочетает в себе два метода: построение словаря по алгоритму LZ77 и последующее кодирование полученных символов и ссылок кодами Хаффмана.

2 Формальная постановка задачи

Задача декодирования DEFLATE заключается в преобразовании непрерывного потока бит (Bitstream) в исходную последовательность байт. Основная сложность заключается в том, что границы данных не совпадают с границами байтов.

Формально процесс можно представить как трехуровневый конвейер:

1. **Битовый уровень:** Чтение сырых данных. Важной особенностью RFC 1951 является порядок LSB (Least Significant Bit): первым читается самый младший бит байта.
2. **Символьный уровень (Хаффман):** Интерпретация последовательностей бит как кодов. На выходе мы получаем либо одиночный байт (литерал), либо команду на копирование из прошлого.
3. **Структурный уровень (LZ77):** Восстановление данных с помощью «скользящего окна». Если получена команда копирования, мы заглядываем в уже распакованный текст и дублируем фрагмент.

3 Грамматика и структура потока

Для описания формата удобно использовать контекстно-свободную грамматику. Если говорить простыми словами, поток данных — это цепочка «блоков», где каждый блок сообщает, как именно он сжат.

3.1 Высокоуровневая структура

Поток данных описывается следующим образом:

- **DeflateStream** $\rightarrow$ **Block*** + **FinalBlock**
(Поток состоит из любого количества обычных блоков и одного завершающего).
- **Block** $\rightarrow$ **Header** + **Data**
(Заголовок всегда занимает 3 бита: 1 бит — метка конца, 2 бита — тип сжатия).

3.2 Типы блоков (BTYPE)

Алгоритм выбирает один из трех сценариев обработки данных:

Тип	Название	Объяснение
00	Uncompressed	Данные просто лежат «как есть». Мы пропускаем текущие биты до границы байта и копируем массив.
01	Static Huffman	Используется «алфавит», заранее прописанный в стандарте. И отправитель, и получатель знают коды заранее.
10	Dynamic Huffman	Самый сложный режим. Сначала передается «инструкция» (дерево), как расшифровать этот конкретный блок, а затем сами данные.

3.3 Грамматика сжатых данных (LZ77)

Когда мы читаем данные внутри блока (BTYPE 01 или 10), мы сталкиваемся с **алфавитом**, который включает в себя значения от 0 до 285:

- **0–255:** Обычные символы (литералы). Просто выводим байт.
- **256:** Маркер конца блока (End-of-Block). Остановка парсера.
- **257–285:** Коды длины (Length). Означают: «Возьми данные, которые были раньше». После такого кода всегда идет второй код — **Distance** (на сколько байт назад нужно отступить).

Пример: Если в потоке встретилась пара (Length: 5, Distance: 10), это читается так: «Отступи назад на 10 символов и скопируй оттуда 5 символов в текущее место». Именно так достигается сжатие: вместо повторения длинных слов мы записываем короткие ссылки на их прошлые упоминания.

4 Описание алгоритмов трансформации

Для реализации полноценного декодирования в проекте применены следующие алгоритмы:

4.1 Построение канонического дерева Хаффмана

В формате DEFLATE деревья Хаффмана передаются через массив длин кодов (code lengths). Алгоритм восстановления дерева:

1. Подсчет количества кодов каждой длины.
2. Определение численного значения первого (минимального) кода для каждой длины.
3. Последовательное присваивание значений кодам одной длины.

4.2 Декомпрессия LZ77

После декодирования символа Хаффмана, он интерпретируется либо как литерал (байт данных), либо как пара (*length*, *distance*).

- Если это литерал — он просто добавляется в выходной поток.
- Если это ссылка — алгоритм обращается к «скользящему окну» (истории уже распакованных данных), отступает назад на *distance* байт и копирует *length* байт в текущую позицию.

5 Реализация на языке Haskell

Архитектура разработанного декодера спроектирована без использования внешних библиотек парсерных комбинаторов (таких как `Parsec`). Вместо этого реализован собственный легковесный и эффективный стек на базе стандартных монадических трансформеров. Логика системы декомпрессии разделена на три автономных уровня:

1. **Битовый парсер (модуль `Data.BitStream`):** Предоставляет абстракцию `BitReader` для чтения произвольного количества бит из потока, корректно обрабатывая границы байт и порядок считывания от младшего бита к старшему (LSB). Внутреннее состояние (`BitState`) и обработка ошибок инкапсулированы вручную.
2. **Декодер Хаффмана (модуль `Data.Huffman`):** Содержит функции для построения канонического дерева Хаффмана по таблице длин, а также функцию `decodeSymbol`, которая принимает структуру дерева и побитово извлекает из потока исходный символ.
3. **Управление состоянием и LZ77 (модуль `Codec.Deflate.Decoder`):** Главный вычислительный слой, объединяющий логику разбора блоков и ведение истории распакованных данных («скользящего окна»).

Вся основная логика декомпрессии выполняется в рамках кастомного монадического трансформера `DecompressM`:

```
1 type DecompressM a = StateT [Word8] (ExceptT String (State BitState)) a
```

Данный стек монад позволяет прозрачно совмещать чтение низкоуровневых битов (трансформатор над базовым `BitState`), безопасный перехват ошибок валидации структуры DEFLATE-блоков (`ExceptT String`), а также высокоуровневое хранение контекста словаря LZ77 в виде списка байтов в обратном порядке (`StateT [Word8]`).

6 Заключение

Реализованный декодер корректно обрабатывает все типы блоков DEFLATE, включая динамические таблицы Хаффмана. Отказ от тяжеловесных внешних зависимостей в пользу кастомного битового парсера на трансформерах монад позволил добиться полного контроля над состоянием разбора, сохранив при этом лаконичность декларативного описания грамматики RFC 1951 средствами языка Haskell.